CO4-AT3 - COMPARATIVE ANALYSIS

192424319-CH.V.RAMAYOGI

Q7. String Matching: LCS vs Greedy

Given

String X: ABCBDAB

String Y: BDCAB

Objective: Compute the Longest Common Subsequence (LCS) using Dynamic Programming, compare it with a greedy matching attempt, analyze why greedy fails, and explain optimal substructure.

1. Dynamic Programming Approach

Let L[i][j] represent the length of the LCS of X[1..i] and Y[1..j].

If X[i] = Y[j]: L[i][j] = L[i-1][j-1] + 1

If X[i] ≠ Y[j]: L[i][j] = max(L[i-1][j], L[i][j-1])

Base condition: L[i][0] = L[0][j] = 0.

2. DP Table

	Ø	B	D	C	A	B
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	2	3
D	0	1	2	2	2	3
A	0	1	2	2	3	3
B	0	1	2	2	3	4

Therefore, L[7][5] = 4. The LCS length is 4.

3. Finding the LCS by Backtracking

Starting from L[7][5] = 4, backtracking through matching characters gives:

- B = B → select B.
- A = A → select A.
- C = C → select C.
- B = B → select B.

Reading the selected characters in reverse gives BCAB.

One valid optimal LCS is: BCAB

4. Greedy Matching Attempt

A simple greedy strategy scans the first string from left to right and immediately selects the first available matching character in the second string.

X character	Matching character in Y	Greedy decision
A	A	Select A
B	B	Cannot select because B occurs before A
C	C	No suitable position after A
B	B	No suitable B after A
D	D	No suitable D after A
A	—	Already passed
B	—	No match

Depending on the exact greedy rule, the algorithm may obtain a shorter common subsequence such as A. The important point is that a greedy choice can consume an early matching character and prevent a better future sequence.

5. Why Greedy Fails

The main problem is local decision-making. A greedy algorithm may immediately select A because A occurs in both strings. However, after choosing A, there is no useful continuation in the remaining portion of Y that produces the optimal length 4.

The optimal solution instead uses B → C → A → B, giving BCAB.

Thus, a locally attractive choice does not necessarily lead to the globally optimal LCS.

6. Comparison: DP vs Greedy

Feature	Dynamic Programming	Greedy
Decision type	Considers subproblems	Makes immediate local choice
Result	Optimal	Not guaranteed optimal
LCS obtained	BCAB	May produce a shorter sequence
Handles repeated choices	Yes	Can make wrong early commitment
Correctness	Guaranteed optimal	Not guaranteed
Time Complexity	O(mn)	Usually O(m+n) for a simple scan
Space Complexity	O(mn)	O(1) or O(min(m,n))
Suitable for LCS	Yes	No

7. Complexity Analysis

For X = ABCBDAB and Y = BDCAB, m = 7 and n = 5.

Dynamic Programming Time Complexity: O(mn).

Dynamic Programming Space Complexity: O(mn), or O(min(m,n)) with a space-optimized implementation.

Greedy Time Complexity: typically O(m+n) for a simple scan, with O(1) auxiliary space, but it does not guarantee the optimal LCS.

8. Optimal Substructure Insight

LCS has optimal substructure. If X[i] = Y[j], the optimal solution can be constructed from the optimal LCS of the smaller prefixes: L[i][j] = L[i-1][j-1] + 1.

If X[i] ≠ Y[j], the solution is obtained from one of two smaller subproblems: L[i][j] = max(L[i-1][j], L[i][j-1]).

Dynamic Programming uses this property and stores previously calculated results to avoid repeated work.

9. Correctness Analysis

Dynamic Programming is correct because its recurrence considers matching characters and both relevant alternatives when characters do not match. Therefore, it guarantees the maximum possible subsequence length.

For this problem, the optimal LCS length is 4 and one valid LCS is BCAB.

Greedy is not guaranteed to be correct because it does not have the greedy-choice property for general LCS. An early local choice can eliminate a longer subsequence.

10. Final Conclusion

The Dynamic Programming approach produces an optimal LCS of length 4, such as BCAB. It works because LCS has optimal substructure and overlapping subproblems.

A greedy algorithm may be faster and use less memory, but it cannot guarantee the optimal LCS because choosing the earliest or locally best matching character may prevent better choices later.

Key Insight: LCS has optimal substructure, but it does not have the greedy-choice property. Therefore, Dynamic Programming is the appropriate algorithm for solving LCS optimally.